MLA0303-Reinforcement Learning

CO-1 AT-1

Descriptive Experiment

Name: S. Divya

Reg No: 192425163

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To verify the Reinforcement Learning development environment using Gymnasium in Jupyter Notebook.

Algorithm/Procedure

1. Open Jupyter Notebook.
2. Import the Gymnasium library.
3. Create the FrozenLake environment.
4. Print a success message.
5. Verify that the environment is created successfully.

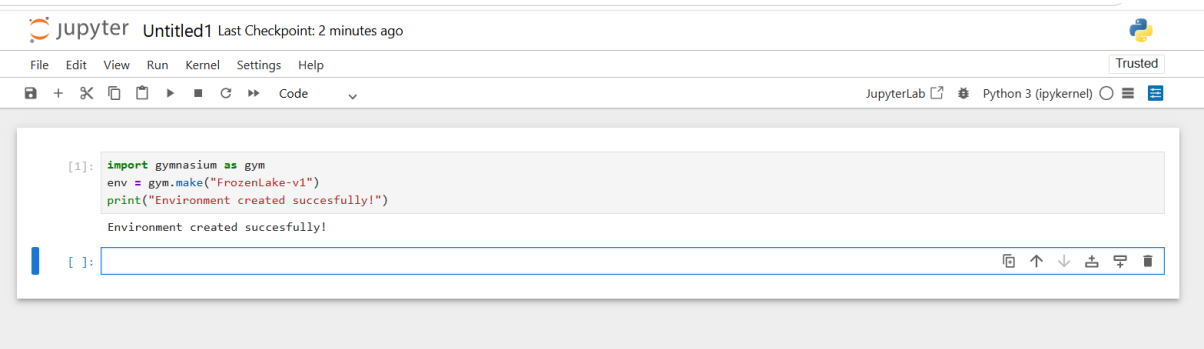
Program

```
import gymnasium as gym


env = gym.make("FrozenLake-v1")


print("Environment created successfully!")
```

Output



Result

The Reinforcement Learning environment was verified successfully.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium

Aim

To create and interact with the FrozenLake environment using Gymnasium.

Algorithm/Procedure

1. Import the Gymnasium library.
2. Create the FrozenLake environment.
3. Reset the environment to get the initial state.
4. Take random actions.
5. Display the state, action and reward.
6. Close the environment.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

print("Initial State:", state)

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)

    print("Step:", i+1)
    print("Action:", action)
    print("Next State:", state)
    print("Reward:", reward)

env.close()
```

Output

```python
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

print("Initial State:", state)

for i in range(5):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)

    print("Step:", i+1)
    print("Action:", action)
    print("Next State:", state)
    print("Reward:", reward)

env.close()
```

```
Initial State: 0
Step: 1
Action: 3
Next State: 0
Reward: 0
Step: 2
Action: 2
Next State: 4
Reward: 0
Step: 3
Action: 1
Next State: 4
Reward: 0
Step: 4
Action: 0
Next State: 4
```

Result

Successfully interacted with the FrozenLake environment.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement a basic Reinforcement Learning framework using an agent and environment.

Algorithm/Procedure

1. Import Gymnasium.
2. Create the FrozenLake environment.
3. Reset the environment.
4. Select random actions.
5. Display the current state, action and reward.
6. Continue until the episode ends.

Program

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

done = False

while not done:
    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("State:", state)
    print("Action:", action)
    print("Reward:", reward)
    state = next_state
    done = terminated or truncated
 env.close()
```

Output

[3]:
```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

done = False

while not done:
    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("State:", state)
    print("Action:", action)
    print("Reward:", reward)

    state = next_state
    done = terminated or truncated

env.close()
```

```
State: 0
Action: 0
Reward: 0
State: 0
Action: 0
Reward: 0
State: 4
Action: 3
Reward: 0
State: 0
Action: 1
Reward: 0
State: 4
Action: 3
```

Result

The Reinforcement Learning framework was implemented successfully.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To simulate a simple Markov Decision Process using states and rewards.

Algorithm/Procedure

1. Define the states.
2. Set the initial state.
3. Move from one state to another.
4. Assign rewards.
5. Stop when the goal state is reached.

Program

```
states = ["Start", "Middle", "Goal"]


current = 0


while current < len(states)-1:
    print("Current State:", states[current])


    reward = 0
    current += 1


print("Current State:", states[current])
print("Reward: 10")
print("Goal Reached")
```

Output

[4]:
```
states = ["Start", "Middle", "Goal"]

current = 0

while current < len(states)-1:
    print("Current State:", states[current])

    reward = 0
    current += 1

print("Current State:", states[current])
print("Reward: 10")
print("Goal Reached")
```

```
Current State: Start
Current State: Middle
Current State: Goal
Reward: 10
Goal Reached
```

Result

The Markov Decision Process was simulated successfully.

Experiment 5: Bellman Equation Demonstration

Aim

To demonstrate the Bellman Equation for calculating the value of a state.

Algorithm/Procedure

1. Define the reward.
2. Define the discount factor.
3. Define the next state value.
4. Apply the Bellman Equation.
5. Display the calculated state value.

Program

```
reward = 5
gamma = 0.9
next_state_value = 10

value = reward + gamma * next_state_value

print("Reward =", reward)
print("Discount Factor =", gamma)
print("Next State Value =", next_state_value)
print("Current State Value =", value)
```

Output

```
[5]: reward = 5
     gamma = 0.9
     next_state_value = 10

     value = reward + gamma * next_state_value

     print("Reward =", reward)
     print("Discount Factor =", gamma)
     print("Next State Value =", next_state_value)
     print("Current State Value =", value)

     Reward = 5
     Discount Factor = 0.9
     Next State Value = 10
     Current State Value = 14.0
```

Result

The Bellman Equation was implemented successfully to calculate the state value.